Activity 2: LoRa-based communication with MQTT integration

Omar Antonio Cornejo Vargas

13 October 2026

1 Introduction

Activity 1 built a telemetry chain in which one ESP32 read a DHT11 sensor and published the readings directly to an MQTT broker over Wi-Fi. Activity 2 adds a LoRa radio link to that chain: a first ESP32 (the *sensor node*) reads the DHT11 and sends the reading over LoRa at 868 MHz, and a second ESP32 (the *gateway*) receives it and publishes it to the MQTT broker. The goals are to implement this link, to study how bandwidth, spreading factor and transmission power affect speed, range, energy and signal quality, and to measure the packet loss over distance.

2 System overview

Figure 1 shows the architecture. Only the two boards and the LoRa link are new: the sensor node no longer needs Wi-Fi, and the gateway is the only board on the network.

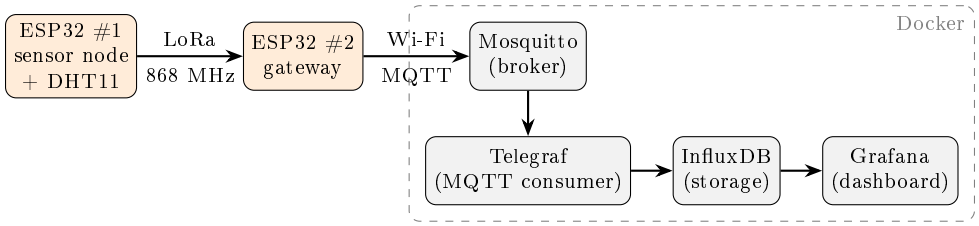


Figure 1: Architecture. Orange: new in this activity. Grey: reused from Activity 1.

A reading changes as follows on its way through the system: the sensor node sends the first JSON over LoRa, the gateway adds the link quality and publishes the second one, and InfluxDB stores the third line in the measurement `lora_weather`:

```
LoRa: {"seq":52,"temperature":26.20,"humidity":72.00}
MQTT: {"seq":52,"temperature":26.20,"humidity":72.00,"rssi":-33,"snr":9.5,"lost":0}
InfluxDB: lora_weather,host=<container> seq=52,temperature=26.2,humidity=72,rssi=-33,snr=9.5,lost=0 <
    ts>
```

The new fields are `seq` (sequence number added by the sender), `rssi` and `snr` (signal strength and signal-to-noise ratio measured by the gateway), and `lost` (lost packets counted by the gateway, Section 4.3).

3 Reused from Activity 1

The server side and the sensor reading were reused from Activity 1 with small changes.

Component	Change in Activity 2
DHT11 reading	Same code (pin 33, `isnan()` validation), now only on the sensor node.
Wi-Fi + MQTT client	Same `PubSubClient` logic, moved to the gateway. New topic `sensor/esp01_lora`.
JSON payload	Same single-object format, with the new fields `seq`, `rssi`, `snr`, `lost`.
`docker-compose.yml`	Same four services. New container names, host ports (1884, 8087, 3001) and database `lora_db`, so both labs can run at the same time.
Telegraf	Same JSON consumer, only `name_override = "lora_weather"` changed. New fields are stored automatically.
Grafana	Same provisioning. New dashboard *LoRa Weather Station* (Figure 2): temperature and humidity, plus RSSI, SNR and QoS panels (received/lost packets, loss rate, sequence number and an OK/LOST status per packet).

Table 1: Parts reused from Activity 1 and the changes made to them.



Figure 2: Grafana dashboard *LoRa Weather Station*.

4 Implementation of the LoRa link

4.1 Hardware and Radio configuration

Both boards are **Heltec WiFi LoRa 32 V2**, which have the LoRa radio already built in. Only the sensor node has the DHT11 connected, to read the temperature and the humidity. Both boards are plugged into the PC by USB, but only to get power. The data only reaches the PC through the gateway, which is the only board connected via Wi-Fi to the MQTT broker running on the PC.

LoRa has no negotiation: the receiver only decodes a packet if it uses **exactly the same parameters** as the transmitter. They are declared as constants at the top of both sketches, so every change of BW or SF requires re-uploading both boards (the TX power only affects the sender).

Parameter	Value
Frequency	868.0 MHz
Bandwidth (BW)	125 / 250 / 500 kHz
Spreading factor (SF)	7 / 9
Coding rate (CR)	4/5
Preamble	8 symbols
Sync word	`0x12`
TX power	7 / 10 / 14 dBm

Table 2: LoRa parameters.

4.2 Sensor node firmware (ESP32 #1)

`sketch_lora_sender.ino`:

1. Starts the DHT11 and the radio with the parameters of Table 2. If `radio.begin()` fails it stops and prints the error code.
2. Every second, reads and validates the DHT11 (same code as Activity 1).
3. Builds the JSON with an increasing sequence number `seq`, which lets the gateway detect lost packets.
4. Sends it with `radio.transmit()`, which blocks until the transmission ends.
5. Prints the time on air (`getTimeOnAir()`) and the estimated energy of the packet (Section 5.2).

4.3 Gateway firmware (ESP32 #2)

`sketch_lora_receiver.ino`:

1. Starts the SPI bus and the radio with the same parameters as the sender.
2. Registers an interrupt routine with `setPacketReceivedAction()` that only sets a flag when a packet arrives, and puts the radio in continuous reception.
3. Connects to Wi-Fi and MQTT with the code reused from Activity 1.
4. When the flag is set, reads the packet with `readData()`. A packet with a wrong CRC is counted as *invalid* and discarded. Otherwise it reads the RSSI and SNR, checks the sequence number (a jump bigger than one adds the missing packets to `lost`), appends `rssi`, `snr` and `lost` to the JSON and publishes it. Then it goes back to reception mode.

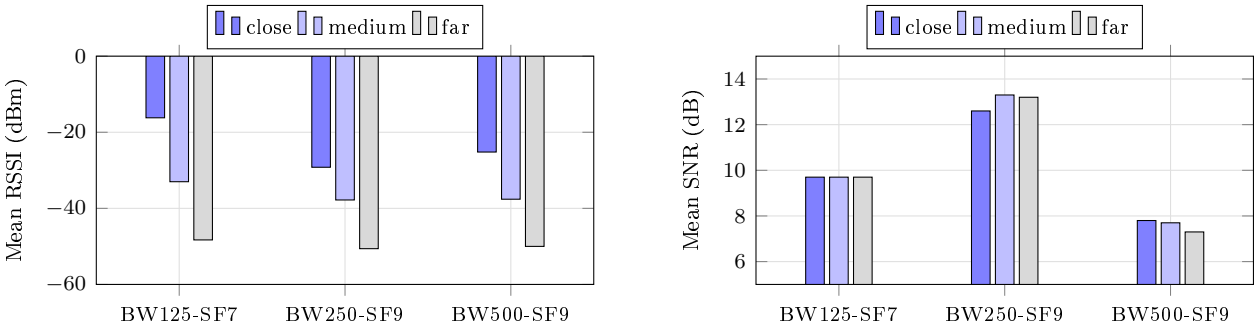
Problem found: at first the loss was estimated by comparing the packet counters of both boards, but they never matched because the boards were restarted at different moments. **Solution:** the sequence number inside the packet, so the gateway calculates the loss on its own.

5 Experiments on LoRa parameters

Common method. The RSSI and SNR of every packet were taken from the Serial monitor of the gateway and saved in `data/lora_samples.csv`. Three reference positions were used inside the room, *close* (a few centimetres), *medium* (normal desk distance) and *far* (the maximum distance the setup allowed, away from the PC). As the boards were placed by hand, the same position can vary a few dB between tests. The RSSI is the received power, the SNR compares the signal with the noise, and LoRa can decode packets with a negative SNR.

5.1 Experiment 1: bandwidth and data rate

In this experiment only the bandwidth (`LORA_BANDWIDTH`) and the spreading factor (`LORA_SPREADING_FACTOR`) were changed, at 14 dBm, and three combinations were tested in the three positions: **BW 125 kHz with SF7**, **BW 250 kHz with SF9** and **BW 500 kHz with SF9**.



(a) Mean RSSI per position.

(b) Mean SNR per position.

Figure 3: Experiment 1: received signal of each BW/SF configuration (`data/bw_sf_summary.csv`).

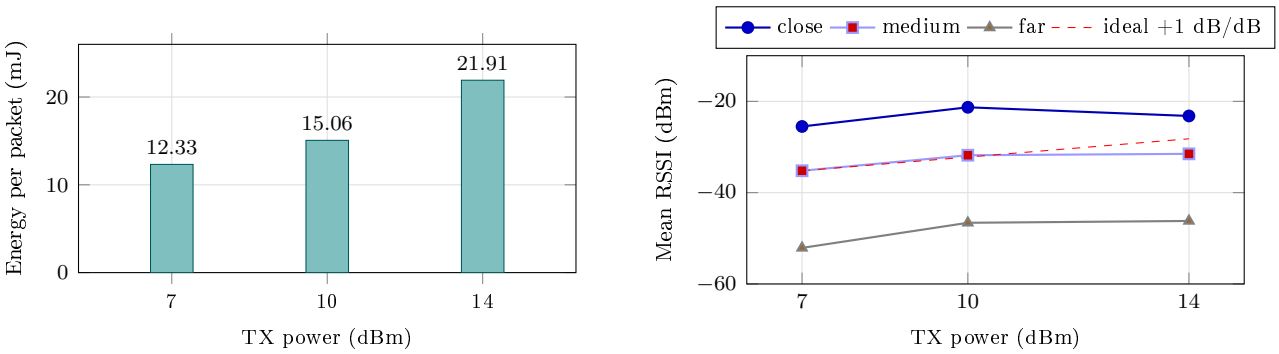
Results and discussion. In theory, a wider bandwidth makes the transmission faster but lets more noise into the channel, so the SNR is lower. A higher SF makes each packet slower but more robust, and a narrower channel with a higher SF gives the best sensitivity and therefore the longest range. During the experiment it was not possible to record every value, but what was observed and the values that could be captured follow this behaviour:

- **Fastest configuration: BW 500 kHz with SF9.** With the widest channel the packets were sent and received faster than with the other two configurations, but they arrived with the most noise: it gave the lowest SNR, about 7.6 dB (Figure 3b).
- **Best signal: BW 250 kHz with SF9.** It was the slowest configuration, but it received the cleanest signal, with the highest SNR (about 13 dB) in all three positions. With a narrower channel and SF9, it is also the configuration with the longest range in theory.
- **BW 125 kHz with SF7** was in the middle, with an SNR of about 9.7 dB.

In short, increasing the bandwidth gives speed, while increasing the SF gives a more robust signal and therefore more range. For a weather station that only sends a few bytes, range usually matters more than speed.

5.2 Experiment 2: transmission power

With BW 125 kHz and SF7 fixed, the sender power was set to 7, 10 and 14 dBm and the RSSI/SNR were recorded in the three positions. No multimeter was available, so the energy per packet is estimated as $E = V \cdot I_{tx} \cdot t_{air}$, with $V = 3.3$ V and $t_{air} = 83$ ms calculated exactly by RadioLib. I_{tx} is the estimated part: the SX1276 datasheet gives 87 mA at 17 dBm and 120 mA at 20 dBm, and the values for 7, 10 and 14 dBm (45, 55, 80 mA) were estimated from them. The absolute energy and the comparison between powers are therefore only valid according to the estimated currents.



(a) Energy per packet ($t_{air} = 83$ ms).

(b) Mean RSSI against TX power.

Figure 4: Experiment 2: energy and received signal for each power (`data/tx_power_summary.csv`).

Results and discussion.

- **Energy.** According to the estimated currents, from 7 to 14 dBm the energy per packet grows 78% (12.3 to 21.9 mJ), while the radiated power is multiplied by 5.
- **Signal strength.** At *medium* and *far* the RSSI improves by 3.7 and 5.9 dB, close to the ideal 7 dB (Figure 4b). At *close* there is no clear trend: with a few centimetres between antennas the receiver is saturated and the RSSI is not reliable. The SNR stays at 9.4–10 dB in all cases.
- **Usefulness.** With a margin of about 70 dB already at 7 dBm, the extra power only spent energy. More power only makes sense near the sensitivity limit.

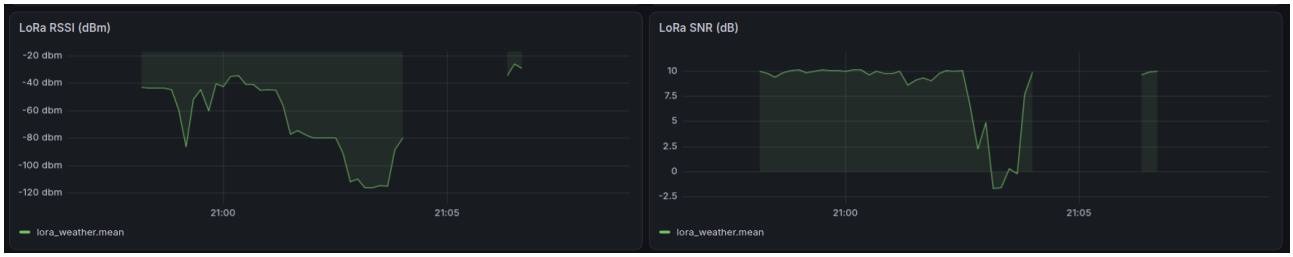


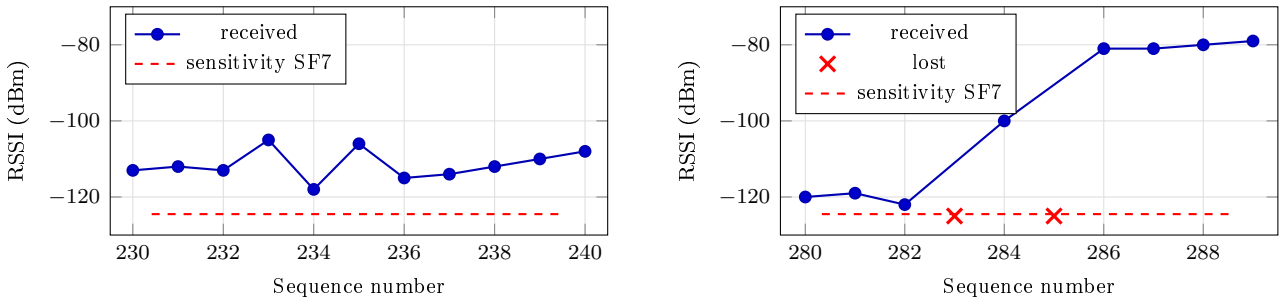
Figure 5: Grafana RSSI and SNR panels over the whole session, including the QoS test of experiment 3. Each step in the curves is a change of configuration, TX power, position or foil. The values of each test are taken from the Serial logs of the gateway.

5.3 Experiment 3: quality of service (packet loss over distance)

With BW 125 kHz, SF7 and 10 dBm, the loss was measured with the sequence number: every jump in `seq` is a packet that never reached the gateway. At the *far* position of the previous experiments the link still had a large margin and lost no packets, and the setup did not allow a longer distance. To bring the link to its limit, the boards were kept at the *far* position and wrapped in aluminium foil, which attenuates the signal as a much longer distance would. Three scenarios were compared: *close* (a few centimetres, no foil), *far + foil (partial)* and *far + foil (full)*. For each one, a window of consecutive packets was taken from the gateway log (`data/qos_packets.csv`).

Scenario	Seq. window	Sent	Received	Lost	Loss (%)	RSSI min / SNR min
Close	10–18	9	9	0	0.0	−28 dBm / 8.8 dB
Far + foil (partial)	230–240	11	11	0	0.0	−118 dBm / −2.7 dB
Far + foil (full)	280–289	10	8	2	20.0	−122 dBm / −5.5 dB

Table 3: Packet loss of each scenario (`data/qos_summary.csv`).



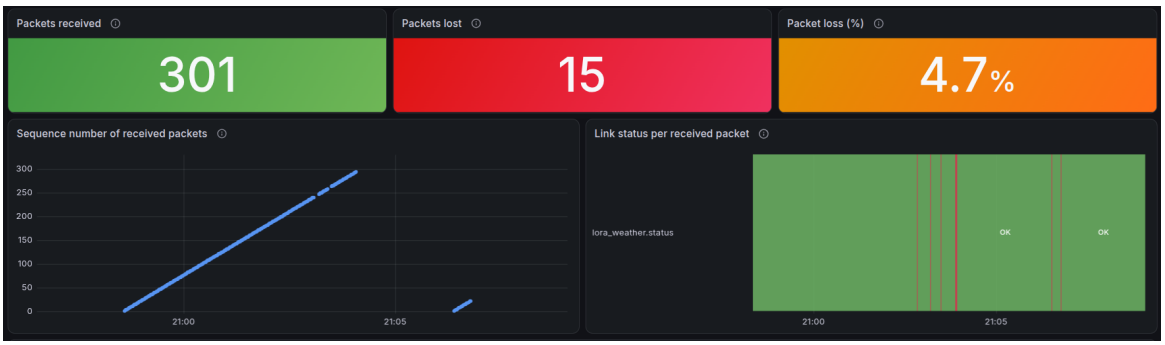
(a) Far + foil (partial): no losses.

(b) Far + foil (full): packets 283 and 285 lost.

Figure 6: Experiment 3: RSSI of each received packet. Dashed line: theoretical sensitivity of SF7 at 125 kHz (−124.5 dBm).

Results and discussion.

- **Close:** no losses, RSSI around −25 dBm, 100 dB above the sensitivity.
- **Far + foil (partial):** the RSSI falls to between −105 and −118 dBm and the SNR becomes negative (−2.7 dB), but no packet is lost (Figure 6a). This is the main advantage of LoRa: it decodes packets below the noise level.
- **Far + foil (full):** the RSSI reaches −122 dBm and the SNR −5.5 dB, only 2.5 dB and 2 dB above the limits (SF7 can decode down to −7.5 dB of SNR). Packets 283 and 285 were lost (Figure 6b). For each one the gateway printed `code -7` (CRC mismatch), so the radio detected the packet but its bits were corrupted. When the foil was removed (packets 286–289) the RSSI went back to about −80 dBm and the losses stopped.
- **Behaviour.** The loss does not grow slowly with distance: the link works perfectly until the signal approaches the sensitivity, and then it fails suddenly.



(a) Loss counters and link status per packet.



(b) Sequence number and packets lost over time.

Figure 7: Grafana QoS panels over the whole session. The counters are accumulated over the entire session, including the tests and the restarts of the sender. The red marks in the link status and the bars of lost packets show when the losses happened.

6 Answers to the guiding questions

How does LoRa compare to Wi-Fi and Bluetooth in terms of range and power consumption? LoRa has a much longer range. Wi-Fi usually reaches 50–100 m indoors and Bluetooth Low Energy 10–50 m, while LoRa can cover several kilometres outdoors because it decodes signals below the noise level. In our test the link still worked at −118 dBm, where Wi-Fi would have disconnected long before. The price is the data rate: a few kbps, against tens of Mbps for Wi-Fi. Regarding power, LoRa only consumes during short bursts and can sleep the rest of the time, while Wi-Fi on the ESP32 has to stay connected to the access point and keep listening, which consumes much more power even when no data is sent. BLE uses even less power, but only over short distances.

What are the trade-offs of using lower data rates in LoRa transmission? A lower data rate makes the receiver more sensitive, so the range and the robustness against noise increase. The drawback is that each packet stays longer in the air, so it is slower and the radio spends more energy on it. A longer time on air also means fewer packets, more collisions with other nodes and more latency. It is the right choice for small, infrequent messages over long distances, but not for sending a lot of data.

How does increasing transmission power affect battery life and interference? More power means more current, so the battery lasts less: in experiment 2, going from 7 to 14 dBm increased the estimated energy per packet by 78%, while the RSSI only improved by 4–6 dB, and with our 70 dB margin that energy was wasted. More power also increases interference: the signal reaches further, so it can collide with other LoRa nodes on the same channel.

7 Conclusions

The LoRa link works end to end: the sensor node sends the DHT11 readings over LoRa, and the gateway adds the link quality and a loss counter and publishes them to the Docker pipeline of Activity 1. The experiments show the three trade-offs of LoRa: bandwidth and spreading factor trade speed for signal quality, transmission power trades energy for signal strength, and the quality of service depends on the margin over the sensitivity.

The code and the data are available at https://github.com/omar-cornejo/EPSEM/tree/main/Iot/Lab2.